

FREE SHIPPING OVER \$50

AMERICA'S BIKE & ADVENTURE SHOP

CONTACT US (USD)

JENSON ^{USA} 

 SEARCH & EXPLORE

 ACCOUNT 



Fueling Growth with **Data**

SQL- Driven Insights into
Sales , Customers & Performance

PRESENTED BY

VAISHNAVI CHAKABORTY

Data Source & Structure

```
1 -- create schemas
2 • CREATE DATABASE Bikes;
```

```
USE Bikes;

-- create tables in the production schema (database)

CREATE TABLE stores (
  store_id INT AUTO_INCREMENT PRIMARY KEY,
  store_name VARCHAR(255) NOT NULL,
  phone VARCHAR(25),
  email VARCHAR(255),
  street VARCHAR(255),
  city VARCHAR(255),
  state VARCHAR(10),
  zip_code VARCHAR(5)
);
```

```
1 • use bikes;
2 • show tables;
3 • select * from brands;
4 • select * from categories;
5 • select * from customers;
6 • select * from order_items;
7 • select * from orders;
8 • select * from products;
9 • select * from staffs;
10 • select * from stocks;
11 • select * from stores;
12
```

Business Questions Addressed

- WHICH PRODUCTS GENERATE THE HIGHEST SALES ACROSS CATEGORIES?
 - WHO ARE THE HIGHEST-SPENDING CUSTOMERS?
 - WHICH STAFF MEMBERS PERFORM ABOVE THE AVERAGE SALES LEVEL?
- WHICH CUSTOMERS HAVE PURCHASED FROM ALL PRODUCT CATEGORIES?
 - WHICH PRODUCTS HAVE NEVER BEEN ORDERED?
 - WHICH PRODUCTS HAVE NEVER BEEN ORDERED?
- HOW DOES CUSTOMER ORDERING BEHAVIOR IMPACT BUSINESS PERFORMANCE?

FIND THE TOTAL NUMBER OF PRODUCTS SOLD BY EACH STORE.

```
-- Find the total number of products
-- sold by each store along with the store name.
select s.store_id,s.store_name,
sum(oi.quantity) total_products
from order_items oi
join orders o
on oi.order_id=o.order_id
join stores s
on o.store_id=s.store_id
group by s.store_id,s.store_name;
```

BALDWIN BIKES LEADS IN SALES VOLUME,
OUTPERFORMING **SANTA CRUZ** AND **ROWLETT BIKES**,
INDICATING STRONGER DEMAND AND STORE
PERFORMANCE.

CALCULATE THE CUMULATIVE SUM OF QUANTITIES SOLD FOR EACH PRODUCT OVER TIME.

```
-- Calculate the cumulative sum of quantities sold for each product over time.
24
25
26 • select * from order_items; -- sum of quantities for each product
27 • select * from orders;
28 • select * from products;
29
30 • select o.order_date,p.product_id,p.product_name ,
31       sum(quantity)
32       over(partition by p.product_id order by o.order_date) total_qty
33 from orders o
34 join order_items od using(order_id)
35 join products p using(product_id);
```

CUMULATIVE SALES FOR PRODUCTS **INCREASE** STEADILY OVER
TIME, INDICATING **CONSISTENT** DEMAND PATTERNS AND
HELPING IDENTIFY PRODUCTS WITH SUSTAINED LONG-TERM
SALES GROWTH.

FIND THE PRODUCT WITH THE HIGHEST TOTAL SALES FOR EACH CATEGORY.

```
-- Find the product with the highest total sales for each category.
select * from categories;
select * from products;
select * from order_items;
with total_Sales as
(select p.product_name,c.category_name,sum(o.quantity*o.list_price-discount) total_sales
from categories c
join products p using (category_id)
join order_items o using(product_id)
group by p.product_name,c.category_name)
select *,rank() over(partition by category_name order by total_Sales desc)as rnk
from total_sales;
```

WITHIN EACH CATEGORY, A SMALL NUMBER OF PRODUCTS DOMINATE TOTAL SALES, SHOWING THAT CATEGORY REVENUE IS HIGHLY CONCENTRATED AROUND TOP-PERFORMING PRODUCTS.

FIND THE CUSTOMER WHO SPENT THE MOST MONEY ON ORDERS.

```
-- Find the customer who spent the most money on orders.
select * from order_items;
select * from orders;
with total_expense as (select o.customer_id,
sum(od.quantity*od.list_price-od.discount) as total_spent
from orders o
join order_items od using(order_id)
group by o.customer_id)
select customer_id,total_spent from total_expense
where total_spent=(select max(total_spent) as max_spent from total_expense);
```

CUSTOMER ID 10 IS THE HIGHEST SPENDER WITH A TOTAL SPEND OF 3,780,159, INDICATING A STRONG CONCENTRATION OF REVENUE CONTRIBUTION FROM A SINGLE HIGH-VALUE CUSTOMER.

FIND THE HIGHEST-PRICED PRODUCT FOR EACH CATEGORY NAME

```
-- Find the highest-priced product for each category name.
select * from products;
select * from categories;
with product_info as (select c.category_name,p.product_id,p.product_name,p.list_price
from products p
join categories c
using(category_id)),
highest_product as (select *,
rank() over (partition by category_name order by p.list_price) as rnk
from product_info)
select distinct * from highest_product
where rnk=1;
```

EACH CATEGORY HAS A PREMIUM PRODUCT, WITH PRICES
RANGING FROM 429 TO 41,699, REFLECTING CLEAR PRICE
DIFFERENTIATION ACROSS CATEGORIES.

FIND THE TOTAL NUMBER OF ORDERS PLACED BY EACH CUSTOMER PER STORE.

```
-- Find the total number of orders placed by each customer per store.
select * from stores;
select * from orders;
select * from order_items;
select o.customer_id,o.store_id,count(o.order_id) as total_orders
from orders o
group by o.customer_id,o.store_id
order by total_orders desc;
```

ORDER FREQUENCY VARIES BY STORE, WITH SOME CUSTOMERS PLACING
UP TO 3 ORDERS AT A SINGLE STORE, INDICATING REPEAT PURCHASING
BEHAVIOR CONCENTRATED AT SPECIFIC LOCATIONS.

FIND THE NAMES OF STAFF MEMBERS
WHO HAVE NOT MADE ANY SALES.

```
-- Find the names of staff members who have not made any sales.
select * from orders;
select * from staffs;

select staff_id,concat(first_name," ",last_name) as staff_name
from staffs
where staff_id Not IN (select staff_id from orders);
```

FOUR STAFF MEMBERS (IDS 1, 4, 5, AND 10) HAVE NOT MADE ANY SALES,
HIGHLIGHTING POTENTIAL UNDERUTILIZATION OR ROLE GAPS WITHIN THE
SALES TEAM.

FIND THE TOP 3 MOST SOLD PRODUCTS IN TERMS
OF QUANTITY.

```
-- Find the top 3 most sold products in terms of quantity.
select * from products;
select * from order_items;

select p.product_name,sum(od.quantity) total_qty
  from products p
 join order_items od
    using (product_id)
 group by p.product_name
 order by total_qty desc
 limit 3 ;
```

SALES VOLUME IS CONCENTRATED AMONG A FEW PRODUCTS, WITH THE TOP THREE
ITEMS SELLING AROUND 290+ UNITS EACH, MAKING THEM THE KEY DRIVERS OF
OVERALL PRODUCT DEMAND.

FIND THE MEDIAN VALUE OF THE PRICE LIST.

```
-- Find the median value of the price list
WITH ranked_prices AS (
  SELECT list_price,
         ROW_NUMBER() OVER (ORDER BY list_price) AS rowno,
         COUNT(*) OVER () AS cnt
  FROM products
)
SELECT round(AVG(list_price),2) AS median_price
FROM ranked_prices
WHERE rowno IN (
  FLOOR((cnt + 1) / 2),
  CEIL((cnt + 1) / 2)
);
```

THE MEDIAN PRODUCT PRICE IS 74,999, INDICATING THAT HALF OF THE PRODUCTS ARE PRICED BELOW THIS VALUE AND HALF ABOVE IT, REFLECTING A MID-TO-PREMIUM OVERALL PRICING STRUCTURE.

LIST ALL PRODUCTS THAT HAVE NEVER BEEN ORDERED.(USE EXISTS)

```
-- List all products that have never been ordered.(use Exists)

select * from products;
select * from order_items;

select p.product_id, p.product_name
from products p
where NOT EXISTS (select * from
order_items od
where od.product_id=p.product_id);
```

FIVE PRODUCTS HAVE NEVER BEEN ORDERED, INDICATING ZERO DEMAND AND POTENTIAL CANDIDATES FOR DISCONTINUATION , REPOSITIONING, OR TARGETED PROMOTIONS.

LIST THE NAMES OF STAFF MEMBERS WHO HAVE MADE
MORE SALES THAN THE AVERAGE NUMBER OF SALES
BY ALL STAFF MEMBERS

IDENTIFY THE CUSTOMERS WHO HAVE ORDERED
ALL TYPES OF PRODUCTS

```
-- List the names of staff members who have made
-- more sales than the average number of sales by all staff members.
with avg_sales as
(select s.staff_id,concat(s.first_name," ",s.last_name) as staff_name ,
round(count(o.order_id),2) as order_count
from staffs s
left join orders o using(staff_id)
group by staff_id ,staff_name)
select staff_id,staff_name from avg_sales
where order_count > (select (round(avg(order_count),0)) from avg_sales )
group by staff_id ,staff_name;
```

FOUR STAFF MEMBERS (IDS 2, 3, 6, AND 7) HAVE MADE MORE SALES
THAN THE OVERALL AVERAGE, INDICATING STRONGER INDIVIDUAL
PERFORMANCE COMPARED TO THE REST OF THE SALES TEAM.

```
-- Identify the customers who have ordered all types of products
select * from categories;
select * from products;
select * from orders;
select * from order_items;
SELECT o.customer_id
FROM orders o
JOIN order_items oi using(order_id)
JOIN products p using(product_id)
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.category_id) = (SELECT COUNT(*) from categories);
```

ONLY ONE CUSTOMER (CUSTOMER ID 9) HAS PURCHASED
PRODUCTS FROM ALL AVAILABLE CATEGORIES, INDICATING
VERY LIMITED FULL-CATEGORY ADOPTION AMONG
CUSTOMERS.

Where Jensen USA Wins Next

Double down on high-performing categories and products

- Categories and products with consistently high sales and repeat demand should get priority in inventory planning, targeted promotions, and premium placement to maximize revenue and reduce stock risk.

Leverage loyal customers and top spenders

- Customers who spend the most and order across multiple categories are prime candidates for loyalty programs, exclusive discounts, and personalized recommendations to increase lifetime value and repeat purchases.



End of the Ride

*Insights
delivered,
Road
ahead
mapped*



Let's Connect



EMAIL :

navyaa394@gmail.com



LINKDIN

[Linkdin](#)



GITHUB :

[Github](#)

